

JOURNAL DE BORD

réalisé par LABORDE Enzo et MAIGNAN Anton.

1 - Création du dataset

Nous avons réalisé un script python qui permettait de sauvegarder les images d'un objet dans des répertoires. Le script met en route la caméra de l'OS qui l'utilise et ainsi propose un menu pour choisir quel type d'objet on veut enregistrer.

- “ESC” pour arrêter
- Objet 1 : on appuie sur “a”
- Objet 2 : on appuie sur “z”
- Objet 3 : on appuie sur “e”
- Fond : on appuie sur “b”
- Autre : on appuie sur “o”

Pour transformer les images en dataset, nous avons alors écrit un second script (*build_dataset.py*) qui agrège toutes les photos capturées par classe en un seul jeu de données exploitable par le modèle. Il parcourt chaque dossier (objet1, objet2, objet3, autre, background), charge les images, les redimensionne en 64x64 pixels, puis les associe à leur label (l'indice de la classe). L'ensemble est ensuite sauvegardé dans un fichier unique *dataset.npz*, sur le même principe que les datasets fournis par Keras, afin de pouvoir le charger directement dans le script d'entraînement avec `np.load`.

Pour la répartition du datasets, nous avons d'abord découpé aléatoirement 80% des images en entraînement et 20% en test. Mais nous avons constaté que le modèle atteignait une accuracy de validation proche de 100% dès les premières epochs, ce qui était suspect. En creusant, nous avons compris que *capture_dataset.py* prend des rafales de 30 photos toutes les 0.4 secondes: deux images voisines d'une même rafale sont donc quasiment identiques (même pose, même éclairage). Un découpage aléatoire faisait fuiter ces quasi-doublons entre l'ensemble d'entraînement et l'ensemble de test, ce qui gonflait artificiellement le score de validation sans que le modèle apprenne réellement à généraliser. Nous avons corrigé cela en remplaçant le découpage aléatoire par un découpage chronologique par classe : les 80% premières photos capturées vont en train, les 20% dernières en test.

2 - Première version de notre modèle

Architecture

Il comprend des couches d'augmentation de données (RandomFlip, RandomRotation, RandomZoom), actives uniquement pendant l'entraînement, pour limiter la mémorisation pure des images vues. Il comprend ensuite 3 blocs de convolution (32, 64 puis 128 filtres, noyaux 3x3, activation ReLU), avec un MaxPooling2D après les deux premiers blocs. La sortie des convolutions est ensuite aplatie avec un Flatten, puis directement reliée à une couche Dense finale à 5 sorties, une par classe, avec une activation softmax.

Résultats

Sur notre dataset, composé de 721 images d'entraînement et 182 images de test réparties sur 5 classes, le modèle atteint une accuracy d'entraînement d'environ 99 % et une accuracy de test d'environ 52 %.

Le modèle converge très rapidement sur les données d'entraînement, en une dizaine d'epochs, alors que la performance sur les données de test reste basse et instable tout au long de l'entraînement.

Preuve par TensorBoard

Les logs générés par le callback TensorBoard pendant l'entraînement confirment quantitativement l'**overfitting** observé.

Sur la courbe epoch_accuracy, l'accuracy d'entraînement atteint 99,3 %, tandis que l'accuracy de validation plafonne à 43-52 % dès l'epoch 10 environ, sans jamais progresser davantage malgré 45 epochs supplémentaires.

Sur la courbe epoch_loss, la loss d'entraînement s'effondre à 0,017, alors que la loss de validation explose au contraire jusqu'à plus de 10 en fin d'entraînement. Cet écart est bien plus marqué qu'avec une architecture régularisée: c'est la signature caractéristique d'un **overfitting**, causé ici par une couche Dense finale démesurée par rapport à la taille du dataset.

En effet, le résumé du modèle montre que la couche Dense compte à elle seule 92 165 paramètres sur 185 415 au total, soit exactement la moitié du modèle. Ce nombre s'explique par le Flatten qui aplatit la dernière carte de features (12x12x128) en un vecteur de 18 432 valeurs directement connecté aux 5 sorties. Avec seulement 721 images d'entraînement pour 185 415 paramètres à apprendre, le modèle a largement la capacité de mémoriser le jeu d'entraînement plutôt que d'apprendre une représentation généralisable.

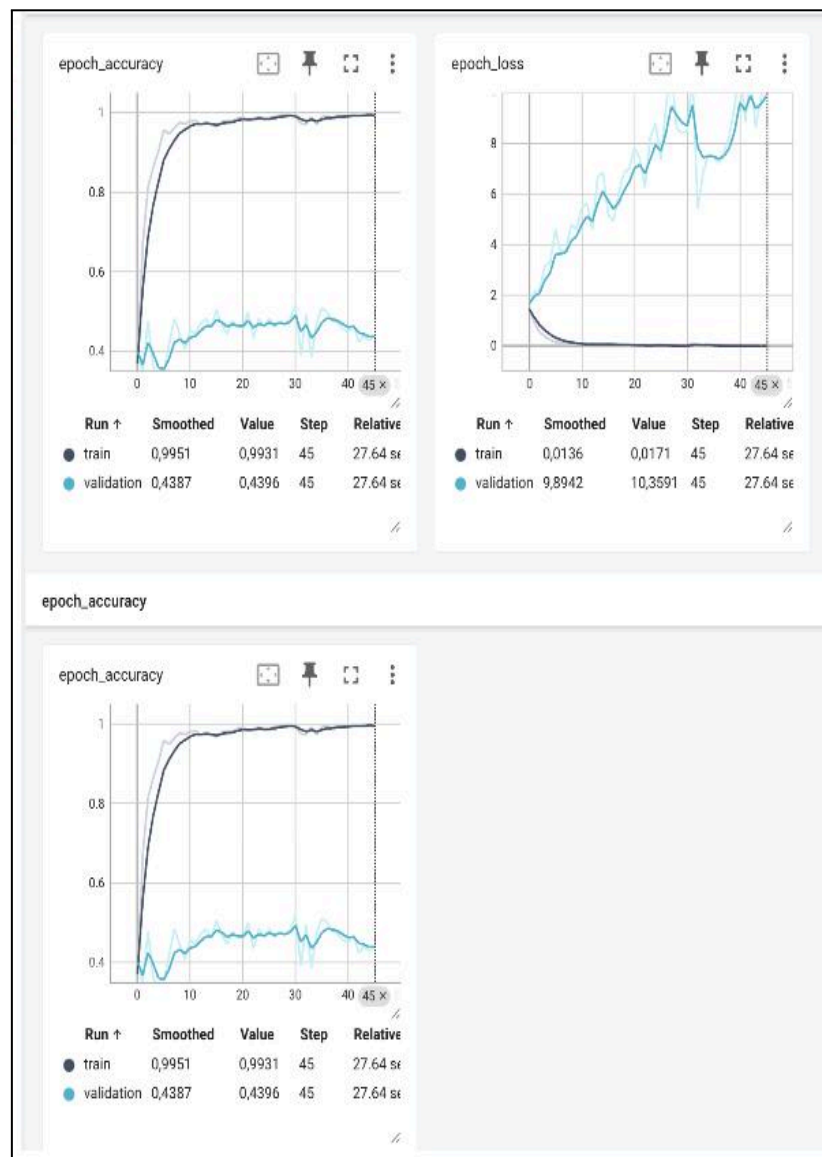


Fig. 1- Graphiques Tensorboard prouvant l'overlifting de la V1

Analyse

L'écart très important entre l'accuracy d'entraînement (99 %) et l'accuracy de test (52 %) montre un overfitting.

La cause principale est la couche Dense finale surdimensionnée : en aplatissant directement la sortie des convolutions sans réduire sa taille, elle expose 92 165 paramètres apprenables à seulement environ 700 images, ce qui permet au modèle de mémoriser des détails spécifiques à chaque image plutôt que d'apprendre la forme des objets.

Conclusion

Le modèle apprend à distinguer les 5 classes avec une accuracy de test qui reste supérieure au hasard, qui serait de 20 % pour 5 classes, mais l'écart avec l'accuracy d'entraînement révèle un surapprentissage sévère.

Cette V1 sert avant tout de démonstration du problème d'overfitting dans ce contexte de couches simples. Ces constats nous amènent à faire évoluer le projet vers une V2.

3 - Deuxième version de notre modèle

Architecture

Pour la V2, nous avons utilisé le transfer learning à partir de ResNet50 de Keras Applications, pré-entraîné sur ImageNet, plutôt qu'un CNN entraîné from scratch comme en V1.

L'entraînement se déroule en deux phases. En phase 1, la base ResNet50 est entièrement gelée (trainable = False) avec par-dessus : couches d'augmentation (RandomFlip, RandomRotation, RandomZoom), suivies de ResNet50, d'un Dropout(0.2), d'un GlobalAveragePooling2D, d'un second Dropout(0.5), puis d'une couche Dense finale à 5 sorties avec activation softmax pour assurer un premier entraînement solide.

En phase 2 (fine-tuning), nous dégelons uniquement les 10 dernières couches de ResNet50, en gardant toutes les couches BatchNormalization gelées pour ne pas casser les statistiques apprises sur ImageNet avec notre petit batch. Le learning rate est réduit à $1e-5$ pour cette phase, contre $1e-3$ en phase 1.

Démarche et itérations

Plusieurs itérations ont été nécessaires pour maîtriser l'overfitting sur cette V2 :

- Un premier essai avec dégel de 30 couches et un seul Dropout(0.3) donnait 76,4% d'accuracy test, mais avec un déséquilibre train/val encore marqué.
- Plusieurs valeurs de Dropout ont été testées empiriquement (0.2 : très overfit ; 0.5 : ratio de loss train/val sain mais accuracy plus faible ; 0.4 : compromis intermédiaire), avant de converger vers la combinaison Dropout(0.2) + Dropout(0.5) actuelle.
- Le levier le plus efficace a été de limiter le dégel du fine-tuning aux dernières couches de ResNet50 plutôt qu'à l'ensemble du réseau : dégeler l'intégralité des 23,6M paramètres sur seulement 721 images d'entraînement provoquait un overfitting sévère (train à 100%, val bloquée, val_loss > 2).

- Nous avons choisi de surveiller `val_loss` plutôt que `val_accuracy` pour l'EarlyStopping : optimiser à répétition sur l'accuracy de validation revient indirectement à sur-ajuster nos choix à ce même jeu de validation, alors que la loss reflète mieux la calibration réelle du modèle sur de nouvelles données.

Résultats

Sur le même dataset (agrégé en résolution 256x256 pour correspondre à l'entrée native de ResNet50), le modèle atteint à la meilleure epoch (epoch 3 de la phase de fine-tuning) :

- Accuracy entraînement : 98,5% / Accuracy test : 79,7%
- Loss entraînement : 0,067 / Loss test : 0,633 (ratio $\times 9,4$)

C'est une nette amélioration par rapport à la V1 (52% de test accuracy), autant en performance brute qu'en écart train/test (18,8 points contre 47 points en V1).

Preuve par TensorBoard

Les courbes `epoch_accuracy` et `epoch_loss` confirment une meilleure généralisation qu'en V1 : la validation se stabilise autour de 0,75-0,80 en accuracy et 0,7-0,9 en loss, sans jamais s'effondrer comme en V1 (où la loss de validation dépassait 10). L'écart avec l'entraînement (qui converge vers 1,0 en accuracy et proche de 0 en loss) reste présent mais nettement plus contenu.

Analyse

Le transfer learning permet au modèle de partir de représentations visuelles déjà riches (appprises sur 1,2M d'images ImageNet) plutôt que de tout apprendre à partir de nos 721 images. Cela explique la nette amélioration par rapport à la V1. L'overfitting résiduel (ratio de loss $\times 9,4$) s'explique par la même cause de fond que sur la V1 : notre dataset reste petit et peu diversifié (une seule séance de capture, même pièce, éclairage similaire), ce qui limite ce qu'un modèle — aussi puissant soit sa base pré-entraînée — peut apprendre de généralisable sur la tête de classification ajoutée.

Conclusion

La V2 démontre l'intérêt du transfer learning et du fine-tuning sélectif face au from scratch sur un petit dataset : +27,7 points d'accuracy test par rapport à la V1, pour un overfitting largement mieux maîtrisé. La limite principale reste la taille et la diversité du dataset, qui plafonne la performance atteignable quelle que soit l'architecture utilisée.